Explicit Nonlinear Structural Dynamics

A FEniCSx Implementation for a Hyperelastic Tube
Under Prescribed Shock Loading

EL SAYED AHMAD Ahmad

Abstract

This report documents the formulation and FEniCSx implementation of an explicit central-difference time integrator applied to a three-dimensional hyperelastic tube subject to prescribed shock kinematics. The material is a compressible Neo-Hookean solid [2]. The spatial discretisation uses linear Lagrangian finite elements within the DOLFINx/UFL framework [3, 4]. The time-stepping scheme follows the Lagrangian structure-preserving formulation of [1]. Each section of the report maps directly to a block of the implementation, explaining both the mathematics and the FEniCSx syntax choices. Solver consistency is verified through an energy balance and a momentum residual check, both embedded in the time loop.

Contents

1 Motivation **1**

2 Continuum Mechanics and Material Model **1**
2.1 Kinematics . . . 1
2.2 Neo-Hookean strain energy . . . 2
2.3 Weak form and internal force . . . 2

3 Function Space and FEniCSx Conventions **2**
3.1 Vectorial P_1 space . . . 2
3.2 Integration measure . . . 3
3.3 Material parameters as `fem.Constant` . . . 3

4 Mass Lumping **3**

5 CFL Time Step **3**

6 Prescribed Boundary Conditions **4**

7 Explicit Newmark Time Integrator **4**

8 Energy Balance Under Prescribed Kinematics **5**
8.1 Kinetic and potential energy . . . 5
8.2 External work and reaction forces . . . 5

9 Momentum Balance Check **6**

10 Validation Results **6**

11 Why Not Implicit? **6**

1 Motivation

Explicit time integration is the natural choice for wave-dominated structural problems: impact, blast, and prescribed shock kinematics. The appeal is algorithmic simplicity — no linear system is solved at each step — combined with an $\mathcal{O}(N)$ cost per step when the mass matrix is lumped to a diagonal. The price is a stability constraint on the time step (CFL condition), but for the wave speeds and mesh sizes typical of soft-tissue or elastomeric structures this bound is rarely more restrictive than the accuracy requirement.

The structure studied here is a hollow hyperelastic tube, clamped at the top and driven at the bottom by a prescribed axial ramp followed by a sinusoidal shock. The nearly incompressible material ($\nu = 0.49$) inflates the dilatational wave speed and tightens the CFL step, but also makes implicit methods unattractive (ill-conditioned tangent stiffness, no stability benefit over explicit at the required accuracy step).

This implementation was written as a personal study of explicit dynamic schemes, using FEniCSx as the assembly engine while keeping the time-stepping loop transparent in Python — a setup directly transferable to a thesis context.

2 Continuum Mechanics and Material Model

2.1 Kinematics

Let $\Omega_0 \subset \mathbb{R}^3$ be the reference domain. The displacement $\mathbf{u}(\mathbf{X}, t)$ maps material points $\mathbf{X} \in \Omega_0$ to their current position. The deformation gradient and its Jacobian are

$$\mathbf{F} = \mathbf{I} + \nabla_{\mathbf{X}}\mathbf{u}, \qquad J = \det \mathbf{F}. \tag{1}$$

The first invariant of the right Cauchy–Green tensor $\mathbf{C} = \mathbf{F}^T\mathbf{F}$ is $I_c = \operatorname{tr} \mathbf{C}$.

2.2 Neo-Hookean strain energy

The compressible Neo-Hookean model [2] is:

$$\psi(\mathbf{F}) = \frac{\mu}{2}(I_c - 3) - \mu \ln J + \frac{\lambda}{2}(\ln J)^2, \tag{2}$$

with Lamé parameters $\mu = E/[2(1+\nu)]$, $\lambda = E\nu/[(1+\nu)(1-2\nu)]$ and $E = 5\,\mathrm{MPa}$, $\nu = 0.49$, $\rho = 1250\,\mathrm{kg/m^3}$.

The model satisfies three physical requirements: the reference configuration is stress-free, $\psi \to +\infty$ as $J \to 0^+$ (no interpenetration), and it recovers linear elasticity for small strains. In the code, ψ is written symbolically in UFL exactly as in (2):

```
I_ufl = ufl.Identity(3)
F_def = I_ufl + ufl.grad(u)
J_det = ufl.det(F_def)
Ic    = ufl.tr(F_def.T * F_def)

psi = (mu/2)*(Ic - 3) - mu*ufl.ln(J_det) + (lam/2)*(ufl.ln(J_det))**2
```

2.3 Weak form and internal force

The total Lagrangian weak form (no body forces, traction-free lateral surface, mass-proportional Rayleigh damping $\mathbf{C} = \alpha\mathbf{M}$) reads: find $\mathbf{u} \in \mathcal{V}$ such that for all $\mathbf{v} \in \mathcal{V}_0$,

$$\rho \int_{\Omega_0} \ddot{\mathbf{u}} \cdot \mathbf{v}\, dV + \alpha\rho \int_{\Omega_0} \dot{\mathbf{u}} \cdot \mathbf{v}\, dV + \int_{\Omega_0} \mathbf{P} : \nabla \mathbf{v}\, dV = 0, \tag{3}$$

where $\mathbf{P} = \partial\psi/\partial\mathbf{F}$ is the first Piola–Kirchhoff stress. Rather than deriving $\mathbf{P}$ by hand, UFL differentiates ψ automatically [3, 4]:

```
f_int_form = fem.form(
    -ufl.derivative(psi * dx, u, ufl.TestFunction(V))
)
```

The call `ufl.derivative(psi*dx, u, v)` produces $\int_{\Omega_0}(\partial\psi/\partial\mathbf{F}) : \nabla\mathbf{v}\, dV$ symbolically. The sign convention `-ufl.derivative` gives the restoring internal force $\mathbf{f}_{int} = -\partial E_{pot}/\partial\mathbf{U}$, consistent with the equation of motion $\mathbf{M}\ddot{\mathbf{U}} = \mathbf{f}_{int}$. This is the key advantage of UFL: changing the constitutive model (Mooney–Rivlin, Ogden, ...) requires replacing only the expression for ψ.

3 Function Space and FEniCSx Conventions

3.1 Vectorial P_1 space

```
V = fem.functionspace(mesh, ("Lagrange", 1, (3,)))

u   = fem.Function(V, name="u")
u_n = fem.Function(V, name="u_n")
v_n = fem.Function(V, name="v_n")
a_n = fem.Function(V, name="a_n")
```

`fem.functionspace` with shape `(3,)` creates a vectorial P_1 space with three scalar components per node, following the FEniCSx tutorial conventions [4]. A `fem.Function` holds both the UFL symbol (used in form expressions) and the underlying PETSc array (`u.x.array`), which can be directly manipulated in NumPy for the explicit update without any additional projection or solve.

3.2 Integration measure

```
dx = ufl.Measure("dx", domain=mesh)
```

The integration measure must be explicitly bound to the mesh. Without `domain=mesh`, UFL cannot locate the mesh during form compilation, and assembled scalar or vector quantities return `NaN` silently — a common source of confusion in FEniCSx codes.

3.3 Material parameters as `fem.Constant`

```
mu  = fem.Constant(mesh, E_mod / (2.0*(1.0 + nu)))
lam = fem.Constant(mesh, E_mod * nu / ((1.0+nu)*(1.0-2.0*nu)))
```

`fem.Constant` wraps a scalar as a UFL expression tied to the mesh. This allows the value to be updated between solves without recompiling the form — useful when sweeping material parameters.

4 Mass Lumping

The consistent mass matrix $M_{ij} = \rho \int N_i N_j\, dV$ is replaced by the row-sum lumped diagonal [1]:

$$(M_L)_{ii} = \sum_j M_{ij}, \qquad (M_L)_{ij} = 0 \text{ for } i \neq j. \tag{4}$$

In FEniCSx this is computed elegantly by multiplying the assembled consistent matrix by the all-ones vector:

```
m_form = fem.form(
    rho * ufl.inner(ufl.TrialFunction(V), ufl.TestFunction(V)) * dx
)
M_mat = petsc.assemble_matrix(m_form)
M_mat.assemble()

ones = fem.Function(V)
ones.x.array[:] = 1.0
M_mat.mult(ones.x.petsc_vec, m_lumped.x.petsc_vec)
m_lumped_inv.x.array[:] = 1.0 / (m_lumped.x.array + 1e-15)
```

The result is a `fem.Function` whose `x.array` contains the lumped masses component-wise. Inversion is then element-wise ($\mathcal{O}(N)$), so $\mathbf{M}_L^{-1}\mathbf{f}$ reduces to a NumPy array multiplication with no linear solve. The small guard `+1e-15` protects ghost DOFs beyond `local_size` that receive zero lumped mass during assembly.

5 CFL Time Step

The central-difference scheme is conditionally stable. The stability limit is controlled by the highest frequency in the semi-discrete system, which is bounded by the dilatational wave speed and the minimum element size [1]:

$$\Delta t = \alpha_s \frac{h_{min}}{c_{mech}}, \qquad c_{mech} = \sqrt{\frac{\lambda + 2\mu}{\rho}}. \tag{5}$$

The safety factor $\alpha_s = 0.3$ accounts for finite-strain stiffening and the near-incompressible bulk modulus inflation ($K/\mu \approx 83$ at $\nu = 0.49$). The minimum edge length is computed in parallel by looping over local cells and reducing across MPI ranks:

```
h_min_local = 1.0e10
for i in range(mesh.topology.index_map(tdim).size_local):
    verts = c_to_v.links(i)
    for a, b in [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]:
        h_min_local = min(h_min_local,
            np.linalg.norm(coords[verts[a]] - coords[verts[b]]))

h_min = comm.allreduce(h_min_local, op=MPI.MIN)
dt    = CFL * h_min / c_mech
```

For this mesh: $c_{mech} \approx 115\,\mathrm{m/s}$, $\Delta t \approx 5.7 \times 10^{-7}\,\mathrm{s}$, 87 811 steps over $T = 50\,\mathrm{ms}$.

6 Prescribed Boundary Conditions

The top face (tag 101) is fully clamped. The bottom face (tag 102) is driven by a linear ramp followed by a sinusoidal shock:

$$z_{bot}(t) = \begin{cases} -\delta_0\,(t/t_r) & t < t_r \\ -\delta_0 - A\sin(2\pi f_0(t - t_r)) & t \geq t_r \end{cases} \tag{6}$$

with $\delta_0 = 15\,\mathrm{mm}$, $t_r = 5\,\mathrm{ms}$, $A = 5\,\mathrm{mm}$, $f_0 = 200\,\mathrm{Hz}$. The ramp prevents an impulsive velocity jump at $t = 0$ that would inject high-frequency noise into an explicit integrator. The y-component is coupled: $y_{bot} = (0.015 + z_{bot})/2$.

DOF arrays for topological BC application:

```
dofs_top = fem.locate_dofs_topological(V, fdim, facet_tags.find(101))
dofs_bot = fem.locate_dofs_topological(V, fdim, facet_tags.find(102))

dofs_bot_x = dofs_bot * 3
dofs_bot_y = dofs_bot * 3 + 1
dofs_bot_z = dofs_bot * 3 + 2
```

`fem.locate_dofs_topological` finds all DOFs whose support intersects the given facet set identified by mesh tag [4]. The component offset $\{0, 1, 2\}$ reflects the block DOF ordering of FEniCSx vectorial spaces: node i owns global DOFs $3i$, $3i+1$, $3i+2$.

7 Explicit Newmark Time Integrator

The central-difference scheme corresponds to Newmark parameters $\beta = 0$, $\gamma = 1/2$ [1]. It is second-order accurate and conditionally stable. Each step $n \to n+1$ proceeds as follows.

Predictor

$$\mathbf{U}^{n+1,*} = \mathbf{U}^n + \Delta t\, \dot{\mathbf{U}}^n + \tfrac{\Delta t^2}{2}\, \ddot{\mathbf{U}}^n. \tag{7}$$

Dirichlet values are imposed directly on the predicted displacement before force assembly.

Acceleration

$$\ddot{\mathbf{U}}^{n+1} = \mathbf{M}_L^{-1}\mathbf{f}_{int}(\mathbf{U}^{n+1}) - \alpha\dot{\mathbf{U}}^n. \tag{8}$$

Velocity corrector

$$\dot{\mathbf{U}}^{n+1} = \dot{\mathbf{U}}^n + \tfrac{\Delta t}{2}\big(\ddot{\mathbf{U}}^n + \ddot{\mathbf{U}}^{n+1}\big). \tag{9}$$

Kinematic correction at BC DOFs. After the corrector, velocity and acceleration at constrained nodes are overwritten with the analytical prescribed values:

```
v_new[dofs_top_all] = 0.0;  a_new[dofs_top_all] = 0.0
v_new[dofs_bot_x]   = 0.0;  a_new[dofs_bot_x]   = 0.0
v_new[dofs_bot_y]   = vy_bot; a_new[dofs_bot_y]  = ay_bot
v_new[dofs_bot_z]   = vz_bot; a_new[dofs_bot_z]  = az_bot
```

This correction is critical and easy to overlook. Without it, the velocity corrector accumulates floating-point residuals at constrained nodes over $\mathcal{O}(10^5)$ steps, causing the effective boundary velocity to drift from its prescription. For fixed nodes ($g = 0$) this is harmless, but for moving Dirichlet conditions the drift is measurable. Note also that `a_new` at BC DOFs is set to the *prescribed acceleration* — not zero — which is required for the energy balance to close correctly (see Section 8).

Ghost DOF communication. In a distributed MPI run, the internal force assembly requires two scatter operations [3]:

```
petsc.assemble_vector(f_int_vec, f_int_form)
# Sum ghost contributions into owned DOFs
f_int_vec.ghostUpdate(addv=PETSc.InsertMode.ADD,
                      mode=PETSc.ScatterMode.REVERSE)
# Broadcast updated values to ghost DOFs
f_int_vec.ghostUpdate(addv=PETSc.InsertMode.INSERT,
                      mode=PETSc.ScatterMode.FORWARD)
```

The `REVERSE` scatter sums contributions from ghost cells into the owning rank; `FORWARD` then broadcasts the corrected values back. Omitting `REVERSE` produces wrong forces at partition boundaries, visible only in multi-rank runs as spurious stress concentrations.

8 Energy Balance Under Prescribed Kinematics

For an undamped system ($\alpha = 0$), the Hamiltonian

$$\mathcal{H}(t) = E_{kin}(t) + E_{pot}(t) - W_{ext}(t) \tag{10}$$

is conserved to within the truncation error of the integrator. Monitoring $\mathcal{H}$ is the most sensitive consistency check available: it simultaneously tests the constitutive assembly, the mass lumping, the BC imposition, and the reaction force computation.

8.1 Kinetic and potential energy

$$E_{kin} = \tfrac{1}{2}\dot{\mathbf{U}}^T \mathbf{M}_L \dot{\mathbf{U}}, \qquad E_{pot} = \int_{\Omega_0} \psi(\mathbf{F})\, dV. \tag{11}$$

In FEniCSx, E_{pot} is assembled as a scalar form (`fem.assemble_scalar`) using the same ψ expression defined in Section 2. Because `u` is already updated to $\mathbf{U}^{n+1}$ before this call, no additional function evaluation is needed.

8.2 External work and reaction forces

For a system with prescribed Dirichlet BCs, the external work is done by the *reaction forces* at constrained DOFs, not by the internal forces directly. At a constrained DOF i, Newton's law gives:

$$[M_L]_{ii}\, \ddot{g}_i = [f_{int}]_i + R_i \quad \Rightarrow \quad \boxed{R_i = [M_L]_{ii}\, \ddot{g}_i - [f_{int}]_i.} \tag{12}$$

The external work increment is then:

$$\Delta W_{ext} = \sum_{i \in \Gamma_D^{mov}} R_i\, \Delta u_i, \tag{13}$$

where the sum is restricted to *moving* DOFs (y and z at the bottom face). Fixed DOFs have $\Delta u_i = 0$ identically, so their reactions — though large due to the Poisson effect at $\nu = 0.49$ — do zero work and are correctly excluded. In code, this is accumulated *every step*:

```
# R = M_L * a_bc - f_int    (eq. 5)
R_z = m_lumped.x.array[dofs_bot_z]*az_bot - f_int_vec.array[dofs_bot_z
    ]
R_y = m_lumped.x.array[dofs_bot_y]*ay_bot - f_int_vec.array[dofs_bot_y
    ]

# dW = R . du  -- must be accumulated every step, not subsampled
W_ext += comm.allreduce(
    np.dot(R_z, u.x.array[dofs_bot_z] - u_prev_z)
  + np.dot(R_y, u.x.array[dofs_bot_y] - u_prev_y),
    op=MPI.SUM)
```

A common implementation mistake is to use $+[f_{int}]_i$ instead of R_i in (13), which gives the wrong sign and double-counts the potential energy. A second mistake is accumulating W_{ext} only at logging intervals: since Δu changes every step, skipping steps introduces a $\sim 10^4\,\%$ drift in the balance.

9 Momentum Balance Check

As an independent check of the spatial assembly and ghost communication, the nodal residual is monitored on free DOFs:

$$\mathbf{r} = \mathbf{M}_L \ddot{\mathbf{U}} - \mathbf{f}_{int} - \mathbf{R}, \tag{14}$$

where $\mathbf{R}$ is assembled from (12) at all BC DOFs. The relative norm $\varepsilon_{mom} = \|\mathbf{r}\|/\|\mathbf{M}_L\ddot{\mathbf{U}}\|$ should be at machine precision for a consistent discretisation. A value above 10^{-6} would indicate a ghost scatter error or a BC imposition inconsistency.

10 Validation Results

Both checks are run with $\alpha = 0$ (undamped) over $T = 50\,\text{ms}$, 87 811 steps, CFL= 0.3.

Table 1: Validation summary.

Check	Metric	Value	Status
Energy balance	$\lvert\Delta\mathcal{H}\rvert/\lvert W_{ext}\rvert$	0.019 %	✓
Momentum residual	$\max_n \|\mathbf{r}\|/\|\mathbf{M}_L\ddot{\mathbf{U}}\|$	1.04×10^{-8}	✓

The 0.019 % energy drift over 87 811 steps confirms that CFL= 0.3 is sufficiently conservative and that the reaction force formula (12) is consistent. The momentum residual at order 10^{-8} (floating-point round-off) confirms correct internal force assembly and ghost DOF communication.

11 Why Not Implicit?

Implicit Newmark ($\beta = 1/4$) is unconditionally stable but requires a nonlinear solve at every step. For this problem three factors make it unattractive: (i) the near-incompressible tangent stiffness is ill-conditioned ($\kappa \sim K/\mu \approx 83$); (ii) accuracy at $f_0 = 200\,\text{Hz}$ requires $\Delta t \sim 10^{-6}\,\text{s}$ regardless of stability, so unconditional stability buys nothing; (iii) 87 000 sparse $\mathbf{K}_T$ assemblies and iterative solves are an order of magnitude more expensive than the explicit loop. The problem is wave-propagation dominated, and explicit integration is the natural choice.

References

[1] K. Carlberg, R. Tuminaro, P. Boggs. Preserving Lagrangian structure in nonlinear model reduction with application to structural dynamics. *SIAM J. Sci. Comput.*, 37(2):B153–B184, 2015. Preprint: `arXiv:1401.8044`

[2] J. Bonet, R.D. Wood. *Nonlinear Continuum Mechanics for Finite Element Analysis*, 2nd ed. Cambridge University Press, 2008.

[3] A. Logg, K.-A. Mardal, G.N. Wells. *Automated Solution of Differential Equations by the Finite Element Method.* Springer, 2012. Open access: https://fenicsproject.org/book

[4] J.S. Dokken. *The FEniCSx Tutorial*, 2023. https://jsdokken.com/dolfinx-tutorial